Data Science Team Training

Stephen D. Turner, Ph.D.

2026-02-22

Table of contents

Preface **4**

I Technical **5**

1 Dashboards **6**
1.1 Dashboard Anatomy . . . 6
1.1.1 YAML Front Matter . . . 6
1.1.2 Rows and Columns . . . 7
1.1.3 Cards . . . 7
1.1.4 Value Boxes . . . 8
1.1.5 Tabsets . . . 8
1.1.6 Loading Data . . . 9
1.1.7 Value Boxes . . . 10
1.1.8 Charts — Static (ggplot2) . . . 11
1.1.9 Charts — Interactive (plotly) . . . 11
1.1.10 Tables (DT) . . . 12
1.2 Previewing and Rendering . . . 13
1.3 Hosting on GitHub Pages . . . 13
1.3.1 Repository Setup . . . 14
1.3.2 Publishing . . . 14

2 Version Control **15**
2.1 GitHub Setup . . . 15
2.1.1 Creating a GitHub Account . . . 15
2.1.2 Adding an SSH Key . . . 15
2.1.3 Cloning a Repository via SSH . . . 16
2.1.4 Pulling and Pushing . . . 16
2.2 Branching and Merging . . . 16
2.2.1 Internal Organizational Workflow . . . 17
2.2.2 Contributing to External Repositories . . . 18
2.3 Git in Positron . . . 19
2.4 Merge Conflicts . . . 20
2.4.1 What conflict markers look like . . . 21
2.4.2 Resolving conflicts . . . 21

3 Coding with AI 23

II Nontechnical 24

4 Data Organization 25

Appendices 26

References 26

A Other resources 27

Preface

This book is reference material for the CSTE Data Science Team Training (DSTT) program provided by project coach Dr. Stephen Turner. This book contains a collection of useful code, resources, workshop material, and other useful guidance collected over several years of serving as a DSTT coach.

Cross-references: and Appendix A.

Part I

Technical

1 Dashboards

Quarto dashboards use `format: dashboard` to create multi-panel HTML data displays — no server required. They're a middle ground between a static report (a single linear document) and a Shiny app (interactive but requires a running R process). Dashboards are ideal for sharing summaries, KPIs, and charts that need to look polished without the overhead of maintaining a server.

The examples in this chapter are drawn from a real dashboard tracking U.S. drug overdose mortality data from the National Vital Statistics System (2015–2023).

> **Note**
>
> **Live demo:** stephenturner.github.io/quarto-dashboard-demo
> Source code: github.com/stephenturner/quarto-dashboard-demo

1.1 Dashboard Anatomy

1.1.1 YAML Front Matter

A minimal Quarto dashboard needs only two things in the front matter: a title and `format: dashboard`.

```yaml
---
title: "My Dashboard"
format: dashboard
---
```

Additional options control the theme, layout behavior, and navigation elements:

```yaml
---
title: "Drug Overdose Mortality"
subtitle: "National Vital Statistics System, 2015-2023"
format:
  dashboard:
    scrolling: false
```

```yaml
    theme: cosmo
date: today
---
```

Setting `scrolling: false` (the default) sizes each row to fill the viewport height, producing a fixed-height layout. Setting `scrolling: true` allows the page to scroll freely, which is useful when the content is too tall to fit on one screen.

1.1.2 Rows and Columns

Quarto dashboards lay out content in rows by default. Level-2 headings (`##`) define rows, and each code cell within a row becomes a card that fills its share of the available width.

```
## Row {height=25%}

[cards here - share the row equally]

## Row {height=75%}

[cards here - taller row for charts]
```

The `{height=}` attribute (as a percentage or pixel value) controls how much vertical space each row takes. To switch to a column-based layout, use `orientation: columns` in the YAML and `{width=}` attributes on headings instead.

1.1.3 Cards

Every executable code cell in a dashboard automatically becomes a card. The cell's output (a plot, a table, text) fills the card. Use the `#| title:` cell option to give the card a header:

```r
#| title: "Monthly Admissions"

ggplot(admissions, aes(x = month, y = n)) +
  geom_col()
```

1.1.4 Value Boxes

Value boxes are compact summary cards ideal for KPIs and headline numbers. They are created with the `#| content: valuebox` cell option. The cell should return a named list with `icon`, `color`, and `value` keys.

```
#| content: valuebox
#| title: "Total Patients"

list(
  icon  = "person-fill",
  color = "primary",
  value = nrow(patients)
)
```

Icons are Bootstrap Icons names (without the `bi-` prefix). Colors can be Bootstrap theme colors (`"primary"`, `"success"`, `"danger"`, `"warning"`, `"info"`) or any CSS color string.

1.1.5 Tabsets

Adding `.tabset` to a row makes each card within that row a tab instead of a side-by-side panel. Each cell's `#| title:` becomes the tab label.

```
## Row {.tabset}

```r
#| title: "Chart"

# plot code
```

```
#| title: "Data"

# table code
```

```
Tabsets are useful when you have multiple views of the same data (e.g., a chart and the unde

## Building a Dashboard {#sec-dashboards-build}

This section walks through building the overdose mortality dashboard step by step. All code 
only; the executable versions live in the demo repository.
```
```

```
### Project Structure

A Quarto dashboard lives in its own project directory with a `_quarto.yml` file and an `index
```

quarto-dashboard-demo/ _quarto.yml index.qmd od-data-clean.csv

```
The `_quarto.yml` sets the project type and default format options:

```yaml
project:
  type: default

format:
  dashboard:
    theme: cosmo
    scrolling: false
```
```

1.1.6 Loading Data

The setup chunk loads packages and reads the data. Compute any summary values you'll reuse across multiple cells here so they're available throughout the document.

```
#| label: setup
#| message: false
#| warning: false

library(readr)
library(dplyr)
library(ggplot2)
library(plotly)
library(DT)

od <- read_csv("od-data-clean.csv")

# Computed once, reused in value boxes and charts
total_state_years <- nrow(od)
year_min          <- min(od$year)
year_max          <- max(od$year)
```

```r
top_state_recent <- od |>
  filter(year == max(year)) |>
  arrange(desc(pod)) |>
  slice(1)
```

The `pod` column is the proportion of all deaths attributable to overdose — the primary metric throughout the dashboard.

1.1.7 Value Boxes

Three value boxes occupy the first row. Each is a separate code cell with `#| content: valuebox` and a `#| title:`. Place them all under the same `## Row` heading so they share the row equally.

```r
## Row {height=20%}
```

```r
#| content: valuebox
#| title: "State-Year Observations"

list(
  icon  = "table",
  color = "primary",
  value = scales::comma(total_state_years)
)
```

```r
#| content: valuebox
#| title: "Years Covered"

list(
  icon  = "calendar-range",
  color = "info",
  value = paste0(year_min, "-", year_max)
)
```

```r
#| content: valuebox
#| title: "Highest OD % (Most Recent Year)"

list(
  icon  = "graph-up-arrow",
  color = "danger",
```

```
  value = paste0(
    top_state_recent$state, " - ",
    scales::percent(top_state_recent$pod, accuracy = 0.1)
  )
)
```

1.1.8 Charts — Static (ggplot2)

A static ggplot2 chart renders as a PNG image inside a card. It works well for PDF exports and environments without JavaScript, but users can't hover for values or zoom in.

```
#| title: "Overdose % Over Time - Selected States"

highlight_states <- c("WV", "DC", "OH", "KY")

od |>
  filter(state %in% highlight_states) |>
  ggplot(aes(x = year, y = pod, color = state, group = state)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 0.1)) +
  scale_x_continuous(breaks = year_min:year_max) +
  labs(
    x     = NULL,
    y     = "Overdose deaths (% of all deaths)",
    color = "State"
  ) +
  theme_minimal(base_size = 13)
```

1.1.9 Charts — Interactive (plotly)

Wrapping a ggplot in `plotly::ggplotly()` converts it to an interactive chart with hover tooltips, zoom, and pan — no layout changes needed.

```
#| title: "Overdose % Over Time - Selected States"

p <- od |>
  filter(state %in% highlight_states) |>
  ggplot(aes(x = year, y = pod, color = state, group = state)) +
  geom_line(linewidth = 1) +
```

```
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 0.1)) +
  scale_x_continuous(breaks = year_min:year_max) +
  labs(
    x     = NULL,
    y     = "Overdose deaths (% of all deaths)",
    color = "State"
  ) +
  theme_minimal(base_size = 13)

ggplotly(p)
```

> **Tip**
>
> Swapping a static ggplot2 chart for `ggplotly()` is a one-line change that requires no modifications to the dashboard layout. Add interactivity where it helps users explore data; keep static charts where simplicity or export quality matters more.

For finer control over tooltips, use the `text` aesthetic and pass `tooltip = "text"` to `ggplotly()`:

```
bar_data |>
  ggplot(aes(
    x    = reorder(state, total_od),
    y    = total_od,
    fill = highlighted,
    text = paste0(state, ": ", scales::comma(total_od))
  )) +
  geom_col() +
  coord_flip()

ggplotly(p2, tooltip = "text")
```

1.1.10 Tables (DT)

`DT::datatable()` produces an interactive HTML table with built-in search, sorting, and pagination.

```
#| title: "Data"

od |>
```

```r
  mutate(pod = scales::percent(pod, accuracy = 0.01)) |>
  rename(
    State          = state,
    Year           = year,
    `Total Deaths` = ndeath,
    `OD Deaths`    = nod,
    `OD %`         = pod
  ) |>
  DT::datatable(
    filter   = "top",
    options  = list(pageLength = 10, scrollX = TRUE),
    rownames = FALSE
  )
```

`filter = "top"` adds a per-column search box above each column header, allowing users to filter by state or year without any server-side code.

1.2 Previewing and Rendering

From inside the project directory:

```r
quarto preview index.qmd   # live-reload preview in browser
quarto render index.qmd    # render to index.html
```

> **Note**
>
> `quarto preview` on a dashboard opens a standalone browser window showing the dashboard itself. If you also have the book open in another preview, they run independently — the dashboard preview is not embedded in the book.

1.3 Hosting on GitHub Pages

Because `format: dashboard` is incompatible with `format: html` (the book format used here), a dashboard cannot be a page in a Quarto book. The cleanest solution is to keep the dashboard in its own GitHub repository and deploy it separately.

1.3.1 Repository Setup

Create a new GitHub repository (e.g., `quarto-dashboard-demo`) and push the project files:

```
git init
git add .
git commit -m "Initial dashboard"
git remote add origin git@github.com:stephenturner/quarto-dashboard-demo.git
git push -u origin main
```

1.3.2 Publishing

`quarto publish gh-pages` renders the project and pushes the output to the `gh-pages` branch. GitHub Pages serves that branch automatically.

```
quarto publish gh-pages
```

After the first publish, GitHub Pages will be available at `https://stephenturner.github.io/quarto-dashboard-demo`. Subsequent publishes update the live site.

> **Tip**
>
> **Automate with GitHub Actions.** Instead of running `quarto publish` manually on your local machine, you can set up a GitHub Actions workflow that re-renders and deploys on every push to `main`. The Quarto documentation provides a ready-to-use workflow YAML — copy it into `.github/workflows/publish.yml` in your repository and GitHub will handle the rest.

2 Version Control

Git is a distributed version control system that tracks changes to files over time, allowing you to review history, revert mistakes, and collaborate without overwriting each other's work. For data science teams, version control is essential for reproducibility (knowing exactly which code produced which results), collaboration (multiple people working on the same project safely), and auditability (a clear record of what changed and why). GitHub is the most widely used platform for hosting Git repositories and adds features like pull requests, code review, and issue tracking on top of Git's core capabilities.

2.1 GitHub Setup

2.1.1 Creating a GitHub Account

Visit github.com and sign up for a free account. The free tier includes unlimited public and private repositories and is sufficient for most team workflows.

2.1.2 Adding an SSH Key

SSH keys let you authenticate with GitHub without entering your username and password on every push or pull. To generate a key:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Accept the default file location (`~/.ssh/id_ed25519`) and optionally set a passphrase. Then copy the public key:

```
cat ~/.ssh/id_ed25519.pub
```

In GitHub, go to **Settings → SSH and GPG keys → New SSH key**, paste the output, give it a descriptive title (e.g., your machine name), and save.

Verify the connection works:

```
ssh -T git@github.com
```

You should see a message like `Hi username! You've successfully authenticated`.

2.1.3 Cloning a Repository via SSH

When cloning a repository, prefer the SSH URL over HTTPS. On any GitHub repository page, click **Code** and select the **SSH** tab to get the URL.

```
git clone git@github.com:org/repo.git
```

Using SSH avoids repeated password prompts and works with SSH agent forwarding for server-based workflows.

2.1.4 Pulling and Pushing

After cloning, your local repository is linked to the remote (called `origin` by default). To sync your local branch with the latest changes from the remote:

```
git pull
```

After committing changes locally, push them to the remote:

```
git push
```

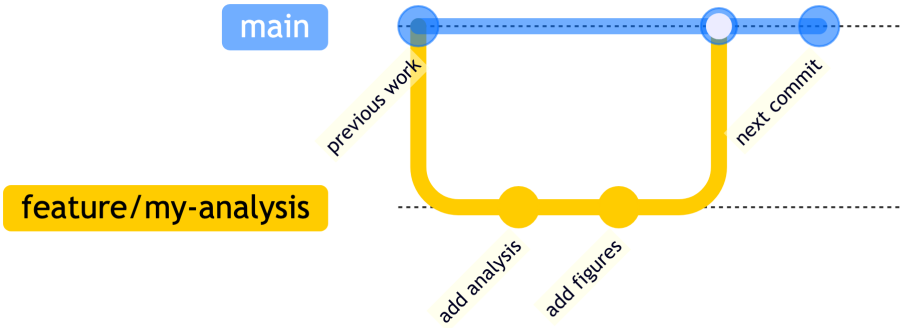
Git tracks the relationship between your local branch and the corresponding remote branch automatically, so these short-form commands will work once the tracking relationship is established.

2.2 Branching and Merging

Branches let multiple people work on different features or fixes simultaneously without interfering with each other. The `main` branch (or `master` in older repositories) typically represents the stable, production-ready state of the project.

2.2.1 Internal Organizational Workflow

In most team workflows, the `main` branch is protected — direct pushes are disabled and changes must go through a pull request (PR) with at least one reviewer. Developers create short-lived branches for each piece of work, then open a PR when ready.



Step 1: Create a branch from `main`

```
git checkout main
git pull
git checkout -b feature/my-analysis
```

Step 2: Make changes, stage, and commit

```
git add analysis.R writeup.qmd
git commit -m "Add initial exploratory analysis for Q1 data"
```

Step 3: Push the branch to GitHub

```
git push -u origin feature/my-analysis
```

The `-u` flag sets the upstream tracking branch so future `git push` and `git pull` commands work without specifying the remote and branch name.

Step 4: Open a pull request

On GitHub, navigate to the repository. A banner will appear prompting you to open a PR from your recently pushed branch. Click **Compare & pull request**, add a description, assign reviewers, and submit.

Step 5: Review, merge, and delete

A reviewer approves the PR and merges it into `main` via the GitHub UI. After merging, delete the branch on GitHub (there is a button on the merged PR page). Locally, clean up with:

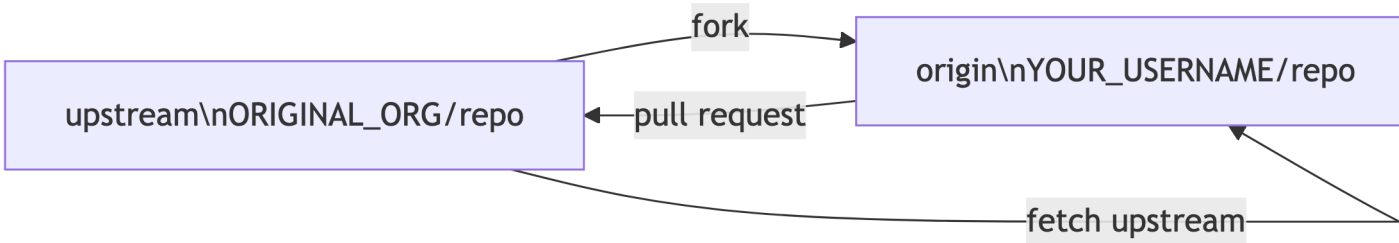
```
git checkout main
git pull
git branch -d feature/my-analysis
```

> **Tip**
>
> **Branch naming conventions** help the team understand what a branch is for at a glance. Common prefixes:
>
> - `feature/` — new functionality (e.g., `feature/survival-analysis`)
> - `fix/` — bug fixes (e.g., `fix/date-parsing-error`)
> - `analysis/` — exploratory or one-off analyses (e.g., `analysis/q2-cohort`)
> - `docs/` — documentation updates

> **Note**
>
> Keep branches short-lived — ideally merged within a few days. Long-running branches diverge significantly from `main`, making merges painful and increasing the likelihood of conflicts.

2.2.2 Contributing to External Repositories

When contributing to a repository you don't have write access to — such as an open-source tool or a public project from another team — the fork workflow is used instead. A fork is a personal copy of the repository under your GitHub account.



Step 1: Fork the repository

On the GitHub repository page, click **Fork** (top right) and choose your account as the destination.

Step 2: Clone your fork

```
git clone git@github.com:YOUR_USERNAME/repo.git
cd repo
```

Step 3: Add the original repository as `upstream`

```
git remote add upstream git@github.com:ORIGINAL_ORG/repo.git
```

Step 4: Branch, work, commit, and push to your fork

```
git checkout -b fix/typo-in-readme
# ... make changes ...
git add README.md
git commit -m "Fix typo in installation section"
git push -u origin fix/typo-in-readme
```

Step 5: Open a pull request to the upstream repository

On your fork's GitHub page, click **Contribute → Open pull request**. This creates a PR from your fork's branch into the original repository's `main` branch.

Step 6: Keep your fork in sync with upstream

As the original repository receives new commits, your fork will fall behind. Sync it with:

```
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

> **Note**
>
> In the fork workflow, `origin` refers to **your fork** and `upstream` refers to the **original repository**. Keeping these straight avoids accidentally pushing to or pulling from the wrong remote.

2.3 Git in Positron

Positron includes a built-in Source Control panel (the branching icon in the left sidebar, or `Ctrl+Shift+G` / `Cmd+Shift+G`) that provides a graphical interface for the most common Git operations.

Viewing changes

The Source Control panel lists all files with uncommitted changes. Files are grouped into **Staged Changes** and **Changes** (unstaged). Click any file to open a diff view showing exactly what was added or removed.

Staging files

Hover over a file and click the `+` icon to stage it, or click the `+` next to the **Changes** heading to stage all modified files at once. To unstage, click the `-` icon next to a staged file.

Committing

Type a commit message in the text box at the top of the Source Control panel and click the **Commit** button (or press `Ctrl+Enter` / `Cmd+Enter`). This is equivalent to `git commit -m "your message"`.

Branching

The current branch name appears in the status bar at the bottom of the window. Click it to open the branch picker, where you can switch to an existing branch or create a new one. This is equivalent to `git checkout` or `git checkout -b`.

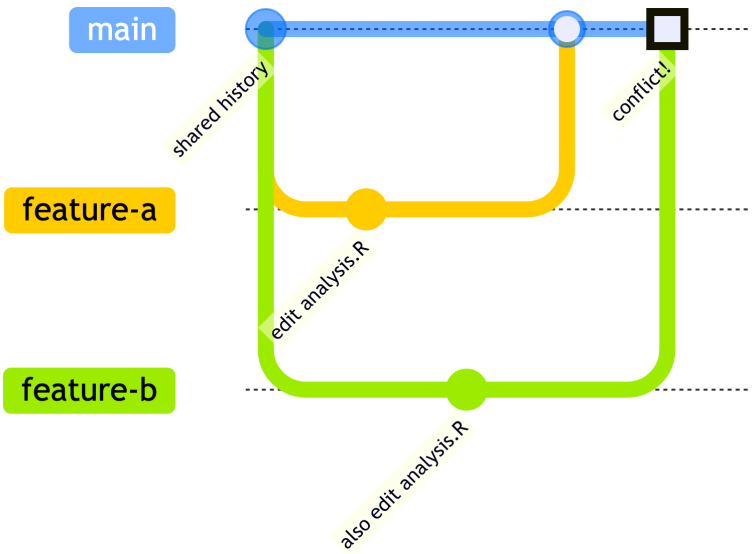
Pushing and pulling

The Source Control panel has **Push** and **Pull** buttons in its toolbar (the `...` menu or the sync icon in the status bar). The sync button performs a pull followed by a push in one action.

> **Tip**
>
> The integrated terminal in Positron (**Terminal → New Terminal**) is always available for Git operations that the UI doesn't expose — such as adding a remote, rebasing, cherry-picking, or any command requiring flags. The UI and terminal work on the same repository state, so you can mix both freely.

2.4 Merge Conflicts

A merge conflict occurs when two branches have made changes to the same lines of the same file, and Git cannot automatically determine which version to keep. Conflicts most commonly arise during `git merge`, `git rebase`, or when accepting a pull request that conflicts with recent changes to `main`.



2.4.1 What conflict markers look like

When a conflict occurs, Git edits the affected file to mark the conflicting regions:

```
<<<<<<< HEAD
result <- model |> predict(new_data)
=======
result <- model %>% predict(new_data)
>>>>>>> feature/update-pipe-syntax
```

- Everything between `<<<<<<< HEAD` and `=======` is the version from your current branch.
- Everything between `=======` and `>>>>>>>` is the version from the branch being merged in.

2.4.2 Resolving conflicts

1. Run `git merge <branch>` (or let a PR trigger the conflict). Git will report which files have conflicts.
2. Open each conflicted file. Positron highlights conflict regions with inline buttons: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**, or **Compare Changes**. Click the appropriate option or edit the file manually to produce the correct final version.
3. After resolving all conflicts in a file, save it and stage it:

```
git add path/to/resolved-file.R
```

4. Once all conflicted files are resolved and staged, complete the merge:

```
git commit
```

Git will pre-populate a commit message describing the merge; you can accept it as-is.

> **Warning**
>
> Never leave conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) in committed code. The file will be syntactically broken and the code will not run. Always verify the conflict is fully resolved before staging.

> **Tip**
>
> `git status` is your best friend during a conflict. It lists which files are in conflict (`both modified`), which are already resolved and staged, and what step to take next (commit or continue a rebase).
>
> ```
> git status
> ```

3 Coding with AI

Part II

Nontechnical

4 Data Organization

References

A Other resources